

WORK ORDER MANAGEMENT SYSTEM

Setup, Installation & Verification Guide

Django + Django REST Framework | Windows evaluation path

Purpose

This handoff document provides a reproducible installation and verification path for the technical evaluator. The standard path uses the bundled SQLite database and does not require Node.js or PostgreSQL.

Submission package includes source code, migrations, demo database, REST API, web UI, automated tests and supporting evaluation documentation.

1. Evaluator Overview

The project is a role-based Work Order Management System for handling customer service requests and technician assignments. It provides a web interface, REST API, workflow controls, audit history, attachments, comments and basic reporting.

Component	Details
Application	Django web application with Django REST Framework
Default database	SQLite (bundled as db.sqlite3 for evaluation)
Python	3.10 or newer
Operating system	Windows 10/11 recommended for this guide
Authentication	Token-based API authentication and role-based UI access
Workflow	OPEN → ASSIGNED → IN_PROGRESS → COMPLETED → CLOSED
Optional database	PostgreSQL is supported through documented DATABASE_URL configuration

2. Prerequisites

Install or have access to the following before starting:

- Python 3.10+ with pip available from the command line.
- Git is optional when using the GitHub ZIP download path; it is required for cloning the repository.
- No Node.js installation is required for the standard evaluator path.
- No PostgreSQL installation is required for the standard evaluator path.

Recommended evaluation path

Download or clone the repository, create a virtual environment, install requirements, keep the bundled db.sqlite3, run migrations, start the server, then perform the verification checklist.

3. Obtain the Project

Option A — Download ZIP (simplest)

1. Open the project repository in GitHub.
2. Click Code → Download ZIP.
3. Extract the ZIP to a local folder, for example C:\WorkOrderProject.
4. Open PowerShell in the extracted project folder — the folder must contain manage.py.

Option B — Clone with Git

```
git clone https://github.com/<your-username>/work-order-management-system.git
cd work-order-management-system
```

4. Install and Configure

Run the following commands one at a time from the project root (the directory containing manage.py).

```
py -m venv .env
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
Copy-Item .env.example .env
python manage.py migrate
```

Expected environment indicator

After activation, PowerShell normally displays (.env) at the beginning of the prompt.

PowerShell activation policy (only if required)

```
Set-ExecutionPolicy -Scope Process Bypass
.venv\Scripts\Activate.ps1
```

5. Database and Demo Data

The repository includes a ready-to-use SQLite database at db.sqlite3. For the fastest evaluation, retain this file and use the bundled demo data.

Recommended path

1. Keep the bundled db.sqlite3 file in the project root.
2. Run python manage.py migrate to confirm that the database schema is current.
3. Start the application with python manage.py runserver.
4. Use the demo accounts in the next section to verify role-based access and workflow.

Optional: recreate demo data

Use the following only when evaluating against an empty database or when a fresh demo dataset is desired:

```
python manage.py seed_demo
```

The seed command is designed to be safe to re-run. Demo accounts use the password Demo@12345.

6. Start the Application

```
python manage.py runserver
```

Then open the following endpoints in a browser:

URL	Purpose
-----	---------

http://127.0.0.1:8000/	Application landing / link list
http://127.0.0.1:8000/app/	Main WorkOrder Pro dashboard
http://127.0.0.1:8000/app/login/	Role-based login
http://127.0.0.1:8000/app/work-orders/	Work order management UI
http://127.0.0.1:8000/app/reports/	Reporting view
http://127.0.0.1:8000/admin/	Django administration
http://127.0.0.1:8000/api/	REST API root / API resources

7. Demo Accounts

Username	Role	Password
admin	Admin / Superuser	Demo@12345
manager1	Manager	Demo@12345
tech1	Technician	Demo@12345
tech2	Technician	Demo@12345
customer1	Customer	Demo@12345
customer2	Customer	Demo@12345

Security note

These credentials are for local technical evaluation only. They are demo credentials and must not be reused in production environments.

8. Recommended Evaluation Flow

The following sequence demonstrates the principal business workflow with minimal navigation:

1. Sign in at /app/login/ as customer1 and create a new service request / work order.
2. Sign in as manager1 and confirm the request is visible.
3. Assign tech1 or tech2 to the work order.
4. Sign in as the assigned technician and progress the work order through the permitted workflow.
5. Verify comments, attachments and history according to the role permissions.
6. Open /app/reports/ as manager1 and verify summary reporting.
7. Open /admin/ as admin to inspect managed records and configuration.

Workflow rule

Status transitions are strictly forward-only: OPEN → ASSIGNED → IN_PROGRESS → COMPLETED → CLOSED.

9. API Verification

Detailed endpoint definitions are provided in docs/API.md. A ready-made Postman collection is available at docs/postman_collection.json.

Obtain a token

```
POST http://127.0.0.1:8000/api/auth/token/
```

Form fields: username, password

Use the token

```
Authorization: Token <token>
```

Use the Postman collection for a structured API smoke test across authentication, customers, service requests, work orders and related operations.

10. Verification and Automated Tests

From the project root, run:

```
python manage.py check
python manage.py test
```

Expected result

The Django system check should report no issues, and the automated test suite should complete successfully with an OK result.

Optional coverage

```
python -m pip install coverage
coverage run manage.py test
coverage report
```

11. Submission Deliverables — Where to Find Them

Evaluator Requirement	Repository Location
1. Source code / Git repository	Repository root, service/, workorders/, manage.py
2. Database / schema information	db.sqlite3, docs/DATABASE.md, docs/schema.sql, service/migrations/
3. README with setup instructions	README.md
4. API documentation	docs/API.md, docs/postman_collection.json
5. Manual test cases	docs/MANUAL_TEST_CASES.md
6. SQL queries	docs/REPORT_QUERIES.sql,

	docs/REPORT_QUERIES_SQLITE.sql
7. Known limitations	docs/LIMITATIONS.md

Supporting documents

- docs/TECHNICAL_EVALUATION.md — implementation and evaluation notes.
- docs/SUBMISSION_CHECKLIST.md — submission completeness checklist.
- docs/GIT_PUSH_GUIDE.md — Git/GitHub upload procedure.

12. Troubleshooting

Issue	Resolution
ModuleNotFoundError: No module named django	Activate .venv and run: <code>python -m pip install -r requirements.txt</code>
PowerShell blocks virtual-environment activation	Run <code>Set-ExecutionPolicy -Scope Process Bypass</code> , then activate .venv again
Port 8000 is already in use	Run <code>python manage.py runserver 8001</code> and use <code>http://127.0.0.1:8001/</code>
Admin page is missing local styling	Confirm .env was created from .env.example, restart the server and hard-refresh the browser
Database appears empty	Confirm db.sqlite3 is present in the project root or run <code>python manage.py seed_demo</code>
API requests return authentication errors	Obtain a token from <code>/api/auth/token/</code> and send <code>Authorization: Token <token></code>

13. Completion Criteria

- `python manage.py check` completes without system-check errors.
- `python manage.py test` completes successfully.
- The application opens in a browser at the local server URL.
- At least one customer → manager → technician workflow is verified end-to-end.
- API authentication and a representative API request are verified.
- The evaluator can access all seven requested submission deliverables from the repository.

Evaluator handoff

This guide is intended to be used together with README.md and the documents in docs/. The repository is the primary source of truth for implementation and supporting evidence.